

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf\_095035a0-e9f\agent-a8b375404e6e08e.jsonl`
- SHA-256: `4955d83031cbde6b2252b0f5a9a7990ecbe8ec9a027b338e7d058b2dbf4f9f49`
- Source modified: `2026-06-13T19:16:13+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf\_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:14:23.991Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"Docstring overstates 'never imports torch' ? torch IS imported at module-load via person_detector's top-level import","severity":"low","file":"backend/pipeline/segmenters/fusion_hybrid.py:13,45,89-92","detail":"The module/method docstrings claim the default (person-OFF) path 'never imports torch / touches the GPU'. Empirically, importing the segmenter registry already pulls `backend.pipeline.detectors.person\_detector` (it is imported transitively for type/help and via fusion_features siblings), and person_detector.py has a top-level `import torch` (line 27) and `import cv2` (line 26 of trajectory_hybrid). I confirmed `import backend.pipeline.segmenters` => torch and cv2 are both in sys.modules. What IS correctly deferred is torchvision + supervision (the heavy model weights) and the GPU work ? those stay lazy (confirmed: torchvision/supervision NOT in sys.modules after registry import). So the lazy `from backend.pipeline.detectors.person\_detector import PersonDetector` inside `\_person\_features` (line 92) is effectively redundant (the module is already loaded), and the 'never imports torch' phrasing is inaccurate on a torch-less box: importing the registry would already fail at `import torch`, independent of the fusion cue. This is pre-existing behavior (wasb_hybrid/tracknet_hybrid/yolo_hybrid already transitively import person_detector/torch at registry load), NOT introduced by this PR ? but the new docstring asserts a stronger guarantee than holds.","fix":"Soften the docstring to 'never imports torchvision/supervision nor touches the GPU' (the accurate, load-bearing claim), or drop the now-redundant lazy import line and just import PersonDetector at the top of _person_features' call site for clarity."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:14:28.923Z)

I'll verify this finding by reading the actual code. Let me start by examining the relevant files.

3. user (2026-06-13T19:14:29.384Z)

```

1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6      shuttle-seeded window via two overridable seams the engine exposes:
7
8      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9      GPU-free, computed from the trajectory we already have); and, only when the person cue
10     is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11     (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12     fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13     never imports torch / touches the GPU.
14     - ``_select_for_handoff`` ? optional served Gate A/B: when
15     ``indexing.fusion.min_crossings``
16     (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
17     are
18     dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19     **Persisted candidates remain the full superset** regardless, so the offline
20     experiment
21     harness can re-score any variant.
22
23     Defaults reproduce the substrate exactly: net_crossings/person features are persisted
24     but
25     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
26     seeds
27     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
28     default
29     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
30     through the
31     experiment harness (EXPERIMENT_HARNESS.md), never silently.
32     """
33     from typing import Any, Dict, List, Optional, Tuple
34
35     from backend.pipeline.segmenters.base import segmenter_registry
36     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
37     from backend.pipeline.detectors.base import DetectorRunner
38     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
39     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
40     from backend.pipeline.detectors import rally_gate
41     from backend.pipeline.detectors import fusion_features as ff
42
43     class FusionSegmenter(TrajectoryHybridSegmenter):
44         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
45         B)."""
46
47         family = "fusion"
48         display_label = "Fusion"
49
50         def __init__(self, db: Any, config: Any):
51             super().__init__(db, config)
52             # Built lazily only if the person cue is enabled (keeps torch/GPU off the
53             default path).
54             self._person: Optional[Any] = None
55
56         @property
57         def name(self) -> str:
58             return "fusion_hybrid"
59
60         @property
61         def description(self) -> str:
62             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
63             rallies, "
64
65             "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,

```

[illegible]

```

109     def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                             window_stats: List[Dict[str, Any]],
111                             idx_cfg: Dict[str, Any]) -> List[int]:
112         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113         (0) keep every window ? identical to the substrate. Persisted candidates are
114         untouched (full superset)."""
115         fcfg = idx_cfg.get("fusion", {})
116         min_crossings = int(fcfg.get("min_crossings", 0))
117         min_players = int(fcfg.get("min_players", 0))
118         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120         keep: List[int] = []
121         for i, st in enumerate(window_stats):
122             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                 continue
124             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                 continue
126             keep.append(i)
127         return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131

```

4. user (2026-06-13T19:14:30.389Z)

```

1     """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3     Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4     the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5     `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6     next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7     later the learned classifier reads its persisted features. Keeping detection here (not
in
8     the segmenter) is what makes "add a cue = register a producer" true.
9
10    Behaviour is unchanged from the in-lined version ? only the home moved:
11    - `lazy_load_model` ? load a permissively-licensed torchvision detector
12    - `make_tracker` ? a fresh supervision ByteTrack per chunk
13    - `detect_and_track` ? one frame ? confident persons with persistent IDs
14    - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15    with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16    that feed the candidate `stats` JSON and the learned fusion features.
17
18    Heavy deps (torchvision, supervision) are imported lazily inside the methods so
importing
19    the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
20    the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
21    (ADR-005; ultralytics' AGPL YOLO was dropped).
22    """
23    import logging
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27    import torch
28
29    from backend.utils.geometry import get_velocity_normalised
30
31    logger = logging.getLogger(__name__)
32
33    # torchvision detection models are trained on COCO where the "person" category is
34    # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?

```

```

35 # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
36 _PERSON_CLASS_ID = 1
37
38 # Permissively-licensed torchvision detectors selectable via config
39 # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
40 _DETECTOR_FACTORIES = {
41     "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
42     "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
43     "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
44 }
45 _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
46
47
48 class PersonDetector:
49     """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
50
51     Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
52     ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
53     min_action_window_duration}``); the model loads lazily on first use.
54     """
55
56     def __init__(self, config: Dict[str, Any]):
57         self.config = config
58         # Dynamic torchvision module (None until lazy_load_model); typed Any so the
59         # `self.model([tensor])` inference call type-checks without a quarantine.
60         self.model: Any = None
61         self.device: Optional[str] = None
62
63     def lazy_load_model(self) -> None:
64         if self.model is not None:
65             return
66
67         # Imported lazily so the module imports without torchvision present and so
68         # the test suite (which pre-sets self.model) never pulls in model weights.
69         from torchvision.models import detection as tv_detection
70
71         idx_cfg = self.config.get("indexing", {})
72         use_gpu = idx_cfg.get("use_gpu", True)
73         self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
74         if use_gpu and not torch.cuda.is_available():
75             logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
76
77         model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
78         if model_name not in _DETECTOR_FACTORIES:
79             logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
80             model_name = _DEFAULT_DETECTOR
81
82         builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
83         logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
84         # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
85         self.model = builder(weights="DEFAULT")
86         self.model.eval()
87         self.model.to(self.device)
88
89     def make_tracker(self, fps: float) -> Any:
90         """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
91
92         Imported lazily so tests can mock this method without supervision installed.
93         """
94         import supervision as sv
95         frame_rate = max(1, int(round(fps)))
96         return sv.ByteTrack(frame_rate=frame_rate)
97

```

```

98         def detect_and_track(self, frame: Any, tracker: Any,
99                               score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
100             """Detect persons in a frame and assign persistent track IDs.
101
102             Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
103             This replaces the old ultralytics ``model.track(...)`` call, which bundled
104             detection AND association: torchvision does the detection, supervision's
105             ByteTrack does the association.
106             """
107             import supervision as sv
108
109             # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
110             rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
111             tensor = torch.from_numpy(rgb).permute(2, 0,
112             1).float().div(255.0).to(self.device)
113             with torch.no_grad():
114                 outputs = self.model([tensor])
115
116             out = outputs[0]
117             boxes = out["boxes"].detach().cpu().numpy()
118             labels = out["labels"].detach().cpu().numpy()
119             scores = out["scores"].detach().cpu().numpy()
120
121             # Keep only confident person detections.
122             mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
123             if not mask.any():
124                 return []
125
126             # ByteTrack requires confidence to be populated (it filters on it internally).
127             dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
128             tracked = tracker.update_with_detections(dets)
129
130             result: List[Tuple[int, Tuple[float, float, float, float]]] = []
131             for i in range(len(tracked)):
132                 tid = tracked.tracker_id[i]
133                 if tid is None:
134                     continue
135                 x1, y1, x2, y2 = tracked.xyxy[i]
136                 result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
137             return result
138
139         def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
140             velocity_thresh: float, merge_gap: float,
141             frame_skip: int = 3,
142             chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
143             """
144             Runs person detection + tracking on a video chunk and extracts high-motion
145             action windows. Returns a list of dicts in the full video's timeframe, each
146             with:
147
148                 start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
149                 track_id_count, chunk_index.
150
151             Args:
152                 frame_skip: Process every Nth frame (1 = every frame). Higher values
153                 dramatically reduce inference cost with minimal quality loss.
154                 chunk_index: Index of the source chunk (recorded on each window for
155                 traceability).
156             """
157             cap = cv2.VideoCapture(chunk_path)
158             if not cap.isOpened():
159                 logger.error(f"Could not open {chunk_path}")
160             return []

```

```

157
158     fps = cap.get(cv2.CAP_PROP_FPS)
159     if fps <= 0:
160         fps = 30.0
161
162     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
163     if frame_width <= 0:
164         frame_width = 1280.0 # Sensible fallback
165
166     # Effective sampling rate after frame-skipping
167     effective_fps = fps / max(frame_skip, 1)
168
169     # A fresh tracker per chunk ? track IDs only need to be consistent within a
170     # chunk (windows are computed per chunk, then merged across chunks by time).
171     tracker = self.make_tracker(effective_fps)
172     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
173
174     # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
175     # so windows can be enriched with motion stats for logging + fine-tuning.
176     active_frames = []
177     frame_idx = 0
178     track_history: Dict[int, Tuple[float, float, float, float]] = {}
179
180     while True:
181         ret, frame = cap.read()
182         if not ret:
183             break
184
185         # Skip frames for performance ? at 30fps, every 3rd frame still
186         # gives ~10 samples/second, plenty for human-scale velocity.
187         if frame_idx % max(frame_skip, 1) != 0:
188             frame_idx += 1
189             continue
190
191         # Detect persons + assign persistent track IDs.
192         detections = self.detect_and_track(frame, tracker, score_thresh)
193
194         frame_peak_velocity = 0.0
195         frame_active_ids = set()
196         current_ids = set()
197
198         for track_id, bbox in detections:
199             current_ids.add(track_id)
200
201             if track_id in track_history:
202                 prev_bbox = track_history[track_id]
203                 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
204                 if velocity > velocity_thresh:
205                     frame_peak_velocity = max(frame_peak_velocity, velocity)
206                     frame_active_ids.add(track_id)
207
208             track_history[track_id] = bbox
209
210         # Prune stale IDs that are no longer visible ? prevents false
211         # velocity spikes when the tracker reassigns an old ID.
212         track_history = {k: v for k, v in track_history.items() if k in current_ids}
213
214         if frame_active_ids:
215             active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
216
217         frame_idx += 1
218
219     cap.release()

```

```

220
221     # Group active frames into continuous windows
222     if not active_frames:
223         return []
224
225     max_gap_frames = int(merge_gap * fps)
226
227     def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
228         velocities = [v for _, v, _ in group]
229         track_ids = set().union(*[ids for _, _, ids in group])
230         return {
231             "start": group[0][0] / fps + chunk_start,
232             "end": group[-1][0] / fps + chunk_start,
233             "active_frame_count": len(group),
234             "peak_velocity": max(velocities),
235             "mean_velocity": sum(velocities) / len(velocities),
236             "track_id_count": len(track_ids),
237             "chunk_index": chunk_index,
238         }
239
240     windows = []
241     group = [active_frames[0]]
242     for af in active_frames[1:]:
243         if af[0] - group[-1][0] <= max_gap_frames:
244             group.append(af)
245         else:
246             windows.append(_finalize(group))
247             group = [af]
248     windows.append(_finalize(group))
249
250     # Filter out extremely short action windows
251     min_window_duration = self.config.get("indexing",
252     {}).get("min_action_window_duration", 2.0)
253     windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
254
255     return windows
256
257 def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
258     frame_skip: int = 3) -> Dict[str, Any]:
259     """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
260     video.
261
262     Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
263     ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
264     discards the
265     per-frame boxes), this returns the raw per-frame detections so the fusion
266     features can
267     align them with the shuttle trajectory. It reads the source by **absolute frame
268     index**
269     (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
270     match
271     the WASB trajectory's ``.frame`` exactly ? that alignment is what
272     ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
273
274     Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
275     "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
276     seek
277     accuracy varies by codec (it lands near the requested keyframe then decodes
278     forward);
279     treat the indices as ±a few frames on real footage ? fine for window-level
280     features.
281
282     """
283     self.lazy_load_model()
284     cap = cv2.VideoCapture(video_path)
285     if not cap.isOpened():

```



```

276         logger.error(f"Could not open {video_path}")
277         return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
278
279     fps = cap.get(cv2.CAP_PROP_FPS)
280     if fps <= 0:
281         fps = 30.0
282     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
283     frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
284     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
285     step = max(frame_skip, 1)
286     tracker = self.make_tracker(fps / step)
287
288     start_frame = max(0, int(start_frame))
289     frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
290     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
291     frame_idx = start_frame
292     while frame_idx <= end_frame:
293         ret, frame = cap.read()
294         if not ret:
295             break
296         if (frame_idx - start_frame) % step == 0:
297             frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
298             frame_idx += 1
299     cap.release()
300     return {"frames": frames, "fps": fps,
301           "frame_width": float(frame_width), "frame_height": float(frame_height)}
302

```

5. user (2026-06-13T19:14:30.562Z)

```

1     from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
2     from backend.pipeline.segmenters.motion import MotionPlaySegmenter
3     from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4     from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
5     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
6     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
7     from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
8
9     __all__ = [
10         "segmenter_registry",
11         "BasePlaySegmenter",
12         "MotionPlaySegmenter",
13         "GeminiPlaySegmenter",
14         "HybridYoloSegmenter",
15         "TrackNetHybridSegmenter",
16         "WasbHybridSegmenter",
17         "FusionSegmenter",
18     ]
19

```

6. assistant (2026-06-13T19:14:35.315Z)

Now let me verify the key empirical claim: does importing `backend.pipeline.segmenters` pull `torch` into `sys.modules`? Let me check the sibling segmenters and the trajectory_hybrid base.

7. user (2026-06-13T19:14:36.295Z)

```

1     """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
2     (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).
3
4     Stage-1 detection now lives in `backend/pipeline/detectors/person_detector.py`
5     (`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
6     can be consumed by the future fusion segmenter; this segmenter just orchestrates

```

```

7     chunking, the AI-provider handoff, and DB population around it. Behaviour is unchanged.
8
9     Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
10    service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
11    detection uses torchvision's permissively-licensed (BSD) detectors and the
12    association/tracking that YOLO.track() used to bundle in is provided by
13    supervision's MIT-licensed ByteTrack.
14
15    The registry key stays "yolo_hybrid" for back-compat with existing configs and the
16    DB `detector` tags, even though no YOLO is involved anymore.
17    """
18    import os
19    import time
20    import logging
21    import concurrent.futures
22    from typing import List, Dict, Any, Tuple
23
24    from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
25    from backend.pipeline.detectors.person_detector import PersonDetector, _DEFAULT_DETECTOR
26    from backend.utils.video import get_video_duration, extract_video_chunk
27    from backend.sports import sport_registry
28    from backend.providers import provider_registry
29
30    logger = logging.getLogger(__name__)
31
32
33    def _fmt_ts(seconds: float) -> str:
34        """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
35        seconds = max(0, int(seconds))
36        h, rem = divmod(seconds, 3600)
37        m, s = divmod(rem, 60)
38        return f"{h:02d}:{m:02d}:{s:02d}"
39
40
41    class HybridYoloSegmenter(BasePlaySegmenter):
42        def __init__(self, db, config):
43            super().__init__(db, config)
44            # Stage-1 person cue (detection + tracking + motion windows) ? a reusable
45            # (fusion P1). This segmenter orchestrates chunking + AI handoff + DB around it.
46            self.person = PersonDetector(config)
47
48            @property
49            def name(self) -> str:
50                return "yolo_hybrid"
51
52            @property
53            def description(self) -> str:
54                return ("Two-stage pipeline: torchvision person detection + ByteTrack for fast "
55                    "candidate filtering, then an AI provider for high-precision semantic
56                    boundaries.")
57
58            def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
59                None, max_frames: int = None) -> List[Dict[str, Any]]:
60                self.person.lazy_load_model()
61
62                # Get sport profile config
63                sport_name = self.config.get("sport", "badminton")
64                try:
65                    sport_profile = sport_registry.get(sport_name)
66                except KeyError as e:
67                    logger.error(str(e))
68                    return []
69
70                # Get AI provider and model config

```

```

69         idx_cfg = self.config.get("indexing", {})
70         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
71         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
72
73         # Get appropriate API key based on the provider
74         api_key_env_var = f"{provider_name.upper()}_API_KEY"
75         api_key = os.environ.get(api_key_env_var)
76         if not api_key:
77             logger.error(
78                 f"{api_key_env_var} environment variable is not set! "
79                 f"To run the {provider_name} segmenter, set your API key. "
80                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
81             )
82             return []
83
84         try:
85             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
86         except KeyError as e:
87             logger.error(str(e))
88             return []
89
90         logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}")
91
92         video_duration = get_video_duration(video_path)
93         if video_duration <= 0:
94             logger.error("Invalid video duration.")
95             return []
96
97         chunk_duration = idx_cfg.get("chunk_duration", 300)
98         chunk_overlap = idx_cfg.get("chunk_overlap", 30)
99         velocity_thresh = idx_cfg.get("yolo_velocity_threshold", 0.15) # Fraction of
frame width per second
100         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
101         frame_skip = idx_cfg.get("yolo_frame_skip", 3)
102         tracker_config = idx_cfg.get("yolo_tracker", "bytetrack.yaml")
103         handoff_padding = idx_cfg.get("ai_handoff_padding",
idx_cfg.get("gemini_handoff_padding", 2.0))
104         ai_max_workers = idx_cfg.get("ai_max_workers", idx_cfg.get("gemini_max_workers",
5))
105         log_candidates = idx_cfg.get("log_yolo_candidates", True)
106         store_candidates = idx_cfg.get("store_yolo_candidates", True)
107
108         # Snapshot of the params that produced these detections ? stored alongside each
109         # candidate so a fine-tuning run can reproduce / correlate against them.
110         detection_params = {
111             "velocity_thresh": velocity_thresh,
112             "merge_gap": merge_gap,
113             "frame_skip": frame_skip,
114             "tracker": tracker_config,
115             "detector_model": idx_cfg.get("detector_model", _DEFAULT_DETECTOR),
116             "detector_score_threshold": idx_cfg.get("detector_score_threshold", 0.5),
117             "min_action_window_duration": idx_cfg.get("min_action_window_duration",
2.0),
118         }
119
120         chunks_dir = os.path.join("output", "chunks_yolo")
121         os.makedirs(chunks_dir, exist_ok=True)
122
123         step = chunk_duration - chunk_overlap
124         chunk_starts = []
125         t = 0.0

```

```

126         while t < video_duration:
127             chunk_starts.append(t)
128             t += step
129
130         num_chunks = len(chunk_starts)
131         logger.info(f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
chunks.")
132
133         all_candidate_windows = []
134
135         t_start = time.time()
136         prefetch_workers = idx_cfg.get("yolo_prefetch_workers", 2)
137
138         if num_chunks > 1 and prefetch_workers > 0:
139             # --- PIPELINED PROCESSING ---
140             # GPU inference must stay sequential (single GPU), but ffmpeg chunk
141             # extraction is pure I/O. We pre-extract upcoming chunks in background
142             # threads so extraction of chunk N+1 overlaps with inference on chunk N.
143             logger.info(f"Running detection Phase 2 with pipelined extraction
({prefetch_workers} prefetch workers)...")
144
145             extract_executor =
concurrent.futures.ThreadPoolExecutor(max_workers=prefetch_workers)
146
147             def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
148                 chunk_end = min(cs + chunk_duration, video_duration)
149                 actual_dur = chunk_end - cs
150                 chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
151                 success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
152                 return (ci, cs, chunk_path, success)
153
154             # Submit all extractions upfront ? they'll queue and run as workers free up
155             extract_futures = [
156                 extract_executor.submit(_extract_chunk, ci, cs)
157                 for ci, cs in enumerate(chunk_starts)
158             ]
159
160             # Process results in order: wait for extraction, then run inference on GPU
161             for future in extract_futures:
162                 ci, chunk_start, chunk_path, success = future.result()
163                 chunk_end = min(chunk_start + chunk_duration, video_duration)
164                 logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
165
166                 if not success:
167                     logger.warning(f"Chunk {ci+1} extraction failed. Skipping.")
168                     continue
169
170                 windows = self.person.extract_action_windows_from_chunk(
171                     chunk_path, chunk_start, velocity_thresh, merge_gap,
172                     frame_skip=frame_skip, chunk_index=ci
173                 )
174                 logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
175                 all_candidate_windows.extend(windows)
176
177                 try:
178                     os.remove(chunk_path)
179                 except Exception:
180                     pass
181
182             extract_executor.shutdown(wait=False)
183         else:
184             # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
185             for ci, chunk_start in enumerate(chunk_starts):

```

```

186         chunk_end = min(chunk_start + chunk_duration, video_duration)
187         actual_dur = chunk_end - chunk_start
188         chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
189
190         logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
191         success = extract_video_chunk(video_path, chunk_start, actual_dur,
chunk_path)
192         if not success:
193             continue
194
195         windows = self.person.extract_action_windows_from_chunk(
196             chunk_path, chunk_start, velocity_thresh, merge_gap,
197             frame_skip=frame_skip, chunk_index=ci
198         )
199         logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
200         all_candidate_windows.extend(windows)
201
202         try:
203             os.remove(chunk_path)
204         except Exception:
205             pass
206
207         # Deduplicate overlapping candidate windows across chunks (chunks overlap by
208         # chunk_overlap, so the same rally can surface in two adjacent chunks).
209         def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
210             fa, fb = a["active_frame_count"], b["active_frame_count"]
211             total = fa + fb or 1
212             return {
213                 "start": a["start"],
214                 "end": max(a["end"], b["end"]),
215                 "active_frame_count": fa + fb,
216                 "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
217                 # Frame-weighted mean so longer sub-windows dominate proportionally.
218                 "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb) /
total,
219                 "track_id_count": max(a["track_id_count"], b["track_id_count"]),
220                 "chunk_index": a["chunk_index"],
221             }
222
223         if all_candidate_windows:
224             all_candidate_windows.sort(key=lambda w: w["start"])
225             merged_windows = [all_candidate_windows[0]]
226             for current in all_candidate_windows[1:]:
227                 last = merged_windows[-1]
228                 if current["start"] <= last["end"]: # Overlap
229                     merged_windows[-1] = _merge_stats(last, current)
230                 else:
231                     merged_windows.append(current)
232             all_candidate_windows = merged_windows
233
234         logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
235         logger.info(f"Total candidate windows: {len(all_candidate_windows)}")
236
237         # Per-rally console log (final, full-video timestamps) for video cross-
verification.
238         if log_candidates:
239             for w in all_candidate_windows:
240                 logger.info(
241                     f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
242                     f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']} "
243                     f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
244                     f"tracks={w['track_id_count']}"

```

```

245         )
246
247     # Persist raw candidates for the fine-tuning dataset. Map window index ->
candidate_id
248     # so confirmed AI segments can be linked back in Phase 4.
249     candidate_ids = [None] * len(all_candidate_windows)
250     if store_candidates:
251         for i, w in enumerate(all_candidate_windows):
252             stats = {
253                 "peak_velocity": w["peak_velocity"],
254                 "mean_velocity": w["mean_velocity"],
255                 "active_frame_count": w["active_frame_count"],
256                 "track_id_count": w["track_id_count"],
257             }
258             # Use peak velocity (capped at 1.0) as a coarse generic confidence
score.
259             confidence = min(1.0, w["peak_velocity"])
260             candidate_ids[i] = self.db.add_candidate_segment(
261                 video_id, detector="yolo_hybrid",
262                 start_time=w["start"], end_time=w["end"],
263                 chunk_index=w["chunk_index"], confidence=confidence,
264                 stats=stats, detection_params=detection_params,
265             )
266
267     try:
268         os.rmdir(chunks_dir)
269     except Exception:
270         pass
271
272     # Phase 3: AI Provider Handoff
273     gemini_segments = []
274     gemini_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
275     os.makedirs(gemini_dir, exist_ok=True)
276
277     logger.info(f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows...")
278
279     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
280         w_start = max(0, window["start"] - handoff_padding)
281         w_end = min(video_duration, window["end"] + handoff_padding)
282         w_dur = w_end - w_start
283
284         w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
285         # Re-encode short handoff clips to avoid keyframe-boundary drift
286         success = extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True)
287         if not success:
288             return []
289
290         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
291         segments, metrics = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur, label=f"Handoff {wi+1}")
292
293         # Coordinate mapping ? shift local timestamps back to full-video time
294         valid_segments = []
295         for segment in segments:
296             try:
297                 segment["start_time"] = float(segment["start_time"]) + w_start
298                 segment["end_time"] = float(segment["end_time"]) + w_start
299                 # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
300                 segment["_candidate_id"] = candidate_ids[wi]
301                 valid_segments.append(segment)
302             except (ValueError, TypeError, KeyError) as e:
303                 logger.warning(f"Dropping segment with unparseable timestamps:

```

```

{segment} ? {e}")
304
305         try:
306             os.remove(w_path)
307         except OSError:
308             pass
309
310         return valid_segments
311
312         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
313             futures = [executor.submit(process_candidate_window, wi, w) for wi, w in
enumerate(all_candidate_windows)]
314             # Iterate in submission order to preserve temporal ordering
315             for future in futures:
316                 gemini_segments.extend(future.result())
317
318         try:
319             os.rmdir(gemini_dir)
320         except OSError:
321             pass
322
323         logger.info(f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic
play segments.")
324
325         # Phase 4: DB Population
326         segments_data = []
327         for segment in gemini_segments:
328             try:
329                 start = float(segment["start_time"])
330                 end = float(segment["end_time"])
331                 dur = end - start
332
333                 shot_count = segment.get("shuttle_exchange_count")
334                 if shot_count is None:
335                     shot_count = max(3, int(dur / 0.8))
336                 else:
337                     try:
338                         shot_count = int(shot_count)
339                     except (ValueError, TypeError):
340                         shot_count = max(3, int(dur / 0.8))
341
342                 metadata = {
343                     "shot_count": shot_count,
344                     "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
345                     "detector": f"yolo-hybrid-{model_name}"
346                 }
347
348                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
349                 if segment_id:
350                     # Link back to the candidate that produced this segment. A candidate
351                     # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
352                     # the last one ? the link table records "this candidate was
confirmed".
353                     candidate_id = segment.get("_candidate_id")
354                     if candidate_id is not None:
355                         self.db.link_candidate_to_segment(candidate_id, segment_id)
356                     segments_data.append({
357                         "id": segment_id,
358                         "start_time": start,
359                         "end_time": end,
360                         "duration": dur,

```

```

361         "metadata": metadata
362     })
363     except Exception as e:
364         logger.warning(f"Skipping invalid segment struct: {e}")
365
366     return segments_data
367
368 # Register the segmenter
369 segmenter_registry.register(HybridYoloSegmenter)
370

```

8. user (2026-06-13T19:14:36.630Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27
28    from backend.pipeline.segmenters.base import BasePlaySegmenter
29    from backend.utils.video import get_video_duration, extract_video_chunk
30    from backend.sports import sport_registry
31    from backend.providers import provider_registry
32    from backend.pipeline.detectors.base import DetectorRunner
33
34    logger = logging.getLogger(__name__)
35
36
37    def _fmt_ts(seconds: float) -> str:
38        seconds = max(0, int(seconds))
39        h, rem = divmod(seconds, 3600)
40        m, s = divmod(rem, 60)
41        return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44    class TrajectoryHybridSegmenter(BasePlaySegmenter):
45        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46        boundaries."""
47
48        #: config section under `indexing` (velocity_threshold lives there), the
49        #: output subdir for trajectories, and the metadata detector-tag prefix.
50        family: str = ""
51        #: human-readable label used in log lines.
52        display_label: str = ""

```



```

52
53 def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
54     """Return (runner, detector-specific detection_params extras)."""
55     raise NotImplementedError
56
57 @staticmethod
58 def _probe_fps_width(video_path: str) -> tuple:
59     """Return (fps, frame_width) for a video, with sensible fallbacks."""
60     cap = cv2.VideoCapture(video_path)
61     fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
62     width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
63     cap.release()
64     return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
65
66 @staticmethod
67 def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
68     """Provider-reported exchange count, else a duration-based estimate.
69
70     One canonical implementation ? the twins had drifted (wasb relied on
71     ``int(None)`` raising into the fallback; same result, by accident).
72     """
73     shot_count = segment.get("shuttle_exchange_count")
74     if shot_count is None:
75         return max(3, int(duration / 0.8))
76     try:
77         return int(shot_count)
78     except (ValueError, TypeError):
79         return max(3, int(duration / 0.8))
80
81 # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-----
82 def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
83                             frame_width: float, video_path: str,
84                             idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
85     """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
86     returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
87     rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
88     features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
89     return {
90         "peak_velocity": cw["peak_velocity"],
91         "mean_velocity": cw["mean_velocity"],
92         "active_frame_count": cw["active_frame_count"],
93         "track_id_count": cw["track_id_count"],
94     }
95
96 def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                         window_stats: List[Dict[str, Any]],
98                         idx_cfg: Dict[str, Any]) -> List[int]:
99     """Indices of the candidate windows that proceed to the AI handoff (and thus may
100     become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101     ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102     Persisted candidates are NOT affected ? they remain the full superset for
offline
103     re-scoring."""
104     return list(range(len(windows)))
105
106 def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                  player_pool: Optional[List[str]] = None,
108                  max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109     # override-ignore: the ABC declares the Result contract (Success|Failure)
110     # but every live segmenter still returns a bare list ? the documented

```

```

111         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112         label = self.display_label
113         # --- Sport profile + AI provider wiring (identical across segmenters) ---
114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
123         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
124
125         api_key_env_var = f"{provider_name.upper()}_API_KEY"
126         api_key = os.environ.get(api_key_env_var)
127         if not api_key:
128             logger.error(
129                 f"{api_key_env_var} environment variable is not set! "
130                 f"To run the {provider_name} handoff, set your API key. "
131                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
132             )
133             return []
134         try:
135             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136         except KeyError as e:
137             logger.error(str(e))
138             return []
139
140         video_duration = get_video_duration(video_path)
141         if video_duration <= 0:
142             logger.error("Invalid video duration.")
143             return []
144         fps, frame_width = self._probe_fps_width(video_path)
145
146         # --- Runner config + windowing thresholds ---
147         runner, runner_params = self._build_runner(idx_cfg)
148
149         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154         log_candidates = idx_cfg.get("log_yolo_candidates", True)
155         store_candidates = idx_cfg.get("store_yolo_candidates", True)
156
157         detection_params = {
158             "detector": self.name,
159             "velocity_thresh": velocity_thresh,
160             "merge_gap": merge_gap,
161             "min_action_window_duration": min_window_duration,
162             **runner_params,
163         }
164
165         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166         ok, msg = runner.healthcheck()
167         if not ok:
168             logger.error("%s WSL environment not ready: %s", label, msg)
169             return []
170         logger.info("%s WSL env OK: %s", label, msg)
171
172         # --- Phase 2: trajectory -> candidate rally windows ---

```

```

173         # Trajectory detectors process the whole video in one pass (they carry
174         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175         t_start = time.time()
176         out_dir = os.path.join("output", self.family)
177         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178         if not csv_path:
179             logger.error("%s produced no trajectory; aborting.", label)
180             return []
181
182         points = runner.parse_trajectory_csv(csv_path)
183         logger.info("Parsed %d trajectory points (%d visible).",
184                     len(points), sum(1 for p in points if p.visible))
185
186         all_candidate_windows = runner.trajectory_to_action_windows(
187             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
189             min_window_duration=min_window_duration, chunk_index=0,
190         )
191         logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192                     label, time.time() - t_start, len(all_candidate_windows))
193
194         if log_candidates:
195             for cw in all_candidate_windows:
196                 logger.info(f"[{label} rally]
197                             {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
198                             f"({cw['end'] - cw['start']:.1f}s) |
199                             frames={cw['active_frame_count']} "
200                             f"peak_v={cw['peak_velocity']:.3f}
201                             mean_v={cw['mean_velocity']:.3f}")
202
203         # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
204         # segmenter adds net_crossings + person-cue features. Computed once, reused for
205         # persistence and the handoff gate below.
206         window_stats = [
207             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
208             idx_cfg)
209             for cw in all_candidate_windows
210         ]
211
212         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
213         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
214         if store_candidates:
215             for i, cw in enumerate(all_candidate_windows):
216                 candidate_ids[i] = self.db.add_candidate_segment(
217                     video_id, detector=self.name,
218                     start_time=cw["start"], end_time=cw["end"],
219                     chunk_index=cw["chunk_index"], confidence=min(1.0,
220                     cw["peak_velocity"]),
221                     stats=window_stats[i], detection_params=detection_params,
222                 )
223
224         if not all_candidate_windows:
225             return []
226
227         # --- Phase 3: AI provider handoff over padded candidate clips ---
228         # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
229         # candidates above stay the full superset ? only the served play_segments are
230         # gated.
231         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
232         idx_cfg)
233
234         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
235         os.makedirs(handoff_dir, exist_ok=True)

```

```

228         logger.info("Phase 3: %s validation on %d candidate windows...",
229                     provider_name, len(handoff_indices))
230
231     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
232         w_start = max(0, window["start"] - handoff_padding)
233         w_end = min(video_duration, window["end"] + handoff_padding)
234         w_dur = w_end - w_start
235         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
236         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
237             return []
238         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
239         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
240                                             label=f"Handoff {wi+1}")
241         valid = []
242         for seg in segments:
243             try:
244                 seg["start_time"] = float(seg["start_time"]) + w_start
245                 seg["end_time"] = float(seg["end_time"]) + w_start
246                 seg["_candidate_id"] = candidate_ids[wi]
247                 valid.append(seg)
248             except (ValueError, TypeError, KeyError) as e:
249                 logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
250             try:
251                 os.remove(w_path)
252             except OSError:
253                 pass
254         return valid
255
256     ai_segments: List[dict] = []
257     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
258         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
259                     for wi in handoff_indices]
260         for future in futures:
261             ai_segments.extend(future.result())
262         try:
263             os.rmdir(handoff_dir)
264         except OSError:
265             pass
266         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267
268     # --- Phase 4: DB population + candidate linking ---
269     segments_data = []
270     for segment in ai_segments:
271         try:
272             start = float(segment["start_time"])
273             end = float(segment["end_time"])
274             dur = end - start
275             metadata = {
276                 "shot_count": self._shot_count_from(segment, dur),
277                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278                 "detector": f"{self.family}-hybrid-{model_name}",
279             }
280             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281             if segment_id:
282                 candidate_id = segment.get("_candidate_id")
283                 if candidate_id is not None:
284                     self.db.link_candidate_to_segment(candidate_id, segment_id)
285             segments_data.append({

```

```

286             "id": segment_id, "start_time": start, "end_time": end,
287             "duration": dur, "metadata": metadata,
288         })
289     except Exception as e:
290         logger.warning("Skipping invalid segment struct: %s", e)
291
292     return segments_data
293

```

9. assistant (2026-06-13T19:14:44.796Z)

Now I have the key evidence. Let me confirm the critical chain:

1. ``backend/pipeline/segmenters/__init__.py`` line 4 imports ``yolo_hybrid``.
2. ``yolo_hybrid.py`` line 25: ``from backend.pipeline.detectors.person_detector import PersonDetector, _DEFAULT_DETECTOR`` ? a top-level import.
3. ``person_detector.py`` line 27: ``import torch`` and line 26: ``import cv2`` ? top-level imports.

So importing the segmenters package transitively pulls ``torch`` and ``cv2`` into ``sys.modules`` via ``yolo_hybrid``. This is pre-existing behavior, NOT introduced by the fusion PR.

Now let me look at the exact docstring claims in `fusion_hybrid.py` to assess whether they overstate the guarantee. The finding cites lines 13, 45, 89-92. Let me re-examine the precise wording.

The docstrings say:

- Line 12-13: "The person path lazily builds ONE ``PersonDetector`` ? so the default path ****never imports torch / touches the GPU****."
- Line 45: "Built lazily only if the person cue is enabled (keeps torch/GPU off the default path)."
- Line 91: "Lazy import so the default (person-OFF) path ****never pulls torch/cv2 through this module****."

Let me verify whether the empirical claim about ``sys.modules`` holds by checking what ``fusion_features.py`` imports (the finding mentions "fusion_features siblings") and confirm the import chain doesn't have some guard. Let me also check whether the env actually can import these (the CLAUDE.md says no torch/cv2 in this container).

10. user (2026-06-13T19:14:46.215Z)

```

1      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3      NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4      math is unit-testable in a GPU-free dev container. The detection (person boxes,
5      shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6      module only turns those signals into a handful of **scale/translation-invariant**
7      features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8      memorise this court/camera and fail to generalise off a small labelled set).
9
10     Conventions:
11     - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
12     - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13         index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14     - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15     - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
16         detection counts ? the player-count-only mode).
17     - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18

```

```

19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21     """
22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31         """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32         (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33         if polygon is None or len(polygon) < 3:
34             return False
35         x, y = point
36         inside = False
37         n = len(polygon)
38         j = n - 1
39         for i in range(n):
40             xi, yi = polygon[i]

```

11. user (2026-06-13T19:14:47.208Z)

```

1     """WASB-based hybrid play segmenter.
2
3     Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4     WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5     badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7     For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8     candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9     min_window_duration) are what get tuned against golden rally labels.
10
11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.
14     """
15     from typing import Any, Dict, Tuple
16
17     from backend.pipeline.segmenters.base import segmenter_registry
18     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19     from backend.pipeline.detectors.base import DetectorRunner
20     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23     class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24         family = "wasb"
25         display_label = "WASB"
26
27         @property
28         def name(self) -> str:
29             return "wasb_hybrid"
30
31         @property
32         def description(self) -> str:
33             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
34                     "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
35
36         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:

```

```

37         cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38         return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
40
41     segmenter_registry.register(WasbHybridSegmenter)
42

```

12. user (2026-06-13T19:14:47.435Z)

```

1     """TrackNet-based hybrid play segmenter.
2
3     Phase 2 (candidate generation) uses a TrackNetV4 *shuttle* trajectory computed
4     in WSL2 (backend/pipeline/detectors/tracknet_runner.py, ADR-001). Rationale: a
5     tiny, fast, motion-blurred shuttle is exactly what general detectors miss; a
6     purpose-built trajectory model localises it far more precisely, giving cleaner
7     candidate rallies than person-velocity heuristics.
8
9     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
10    lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
11    TrackNet runner and identity.
12    """
13    from typing import Any, Dict, Tuple
14
15    from backend.pipeline.segmenters.base import segmenter_registry
16    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
17    from backend.pipeline.detectors.base import DetectorRunner
18    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
19
20
21    class TrackNetHybridSegmenter(TrajectoryHybridSegmenter):
22        family = "tracknet"
23        display_label = "TrackNet"
24
25        @property
26        def name(self) -> str:
27            return "tracknet_hybrid"
28
29        @property
30        def description(self) -> str:
31            return ("Two-stage pipeline: TrackNetV4 shuttle-trajectory detection (WSL2) for
precise "
32                    "candidate rallies + AI provider for high-precision semantic
boundaries.")
33
34        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
35            cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
36            return TrackNetRunner(cfg), {
37                "weights_path": cfg.weights_path,
38                "queue_length": cfg.queue_length,
39            }
40
41
42    segmenter_registry.register(TrackNetHybridSegmenter)
43

```

13. assistant (2026-06-13T19:14:53.691Z)

Now I have the complete picture. Let me verify the empirical sys.modules claim directly. The CLAUDE.md says this container lacks torch/cv2, so a real import would fail ? but I can statically confirm the import chain. Let me check whether torch/cv2 are actually unavailable here, and confirm the import graph by examining whether `fusion\_features` (imported by fusion_hybrid at line 34) pulls person_detector. It does NOT ? fusion_features has no torch/cv2 imports. The torch pull comes from `yolo\_hybrid` (sibling in `\_\_init\_\_.py`).

Let me confirm torch is not installed (which makes the "importing the registry would already fail at import torch" sub-claim verifiable) and double-check there's no lazy-import trick in person_detector's top-level.

14. user (2026-06-13T19:14:59.566Z)

---cv2---

15. assistant (2026-06-13T19:15:02.339Z)

Interesting ? both ``import torch`` and ``import cv2`` succeeded here (no error output). So this dev environment actually does have torch and cv2 available. Let me directly run the empirical test the finding claims: import the segmenters package and check `sys.modules`.

16. user (2026-06-13T19:15:11.349Z)

```
torch in sys.modules: True
cv2 in sys.modules: True
torchvision in sys.modules: False
supervision in sys.modules: False
```

17. assistant (2026-06-13T19:15:21.762Z)

The empirical claim is fully confirmed:

- ``import backend.pipeline.segmenters`` ? ``torch`` and ``cv2`` ARE in ``sys.modules`` (True).
- ``torchvision`` and ``supervision`` are NOT (False) ? those remain genuinely lazy.

This exactly matches the finding's empirical assertions. The chain: ``segmenters/___init___.py`` imports ``yolo_hybrid`` ? ``yolo_hybrid`` top-level imports ``person_detector`` ? ``person_detector`` top-level ``import torch`` and ``import cv2``.

Now let me assess the finding's accuracy claim about the docstring more carefully. The finding's headline is "Docstring overstates 'never imports torch'". Let me look at the precise wording again:

- Line 13: "so the default path ****never imports torch / touches the GPU****." ? This is the strongest claim. Strictly, by the time you reach ``fusion_hybrid``, torch IS already imported (via `yolo_hybrid` sibling). So the default fusion path doesn't ***additionally*** import torch, but torch is already in `sys.modules`. The phrasing "never imports torch" read in isolation is inaccurate on a whole-process basis.
- Line 45: "(keeps torch/GPU off the default path)" ? same issue.
- Line 91: "Lazy import so the default (person-OFF) path ****never pulls torch/cv2 through this module****." ? Note the crucial qualifier: *****through this module*****. This is actually accurate ? the `fusion_hybrid` module itself does not import torch/cv2 at top level; only the lazy ``_person_features`` import does. So line 91's claim is technically correct as worded.

Let me now weigh the nuance. The finding correctly identifies this as low severity, pre-existing (not introduced by the PR), and a docstring-accuracy issue. Let me assess whether it's a nitpick/false-positive vs a real (if minor) doc-accuracy concern.

Now let me evaluate the proposed "fix" sub-claim about redundancy of the lazy import on line 92. The finding says the lazy ``from ... import PersonDetector`` on line 92 is "effectively redundant (the module is already loaded)". Let me verify whether `person_detector` is in `sys.modules` after the registry import.

18. user (2026-06-13T19:15:29.974Z)

```
person_detector in sys.modules: True
```

19. assistant (2026-06-13T19:15:36.496Z)

Confirmed. ``person_detector`` is already in ``sys.modules`` after the registry import (pulled by ``yolo_hybrid``). So the lazy import in ``_person_features`` (line 92) does not save a torch import ? torch is already loaded process-wide. The lazy import only avoids re-binding the name at

module top level, but the heavy work (torch/cv2 load) has already happened transitively.

Let me now do a final adversarial check: is there any way the finding is wrong? The strongest counter-argument is that the docstring on **line 91** specifically says "through this module" ? a precise scoping qualifier that IS accurate. But lines 13 and 45 do not have that qualifier and make the unqualified claim "never imports torch / touches the GPU" and "keeps torch/GPU off the default path". The "touches the GPU" part of those claims IS accurate (no GPU work happens ? torchvision/supervision/model load stays lazy). It's specifically the "imports torch" / "torch off the default path" half that overstates, since torch is already in sys.modules.

Let me verify the git diff to confirm these docstrings are NEW (introduced by this PR), which determines whether the finding is about new-code accuracy vs pre-existing.

20. user (2026-06-13T19:15:38.413Z)

```
M backend/pipeline/detectors/person_detector.py
?? backend/pipeline/segmenters/fusion_hybrid.py
```

21. assistant (2026-06-13T19:15:54.685Z)

`fusion\_hybrid.py` is untracked (`??`) ? it's a brand-new file in this PR. So the docstrings on lines 13, 45, and 91 are all NEW code introduced by this fusion P2 PR. The finding correctly notes the **underlying torch-import behavior** is pre-existing (via yolo_hybrid), but the **docstring assertion** of a stronger guarantee is new.

Let me do my final assessment.

Verdict

The finding is **REAL** but **LOW** severity / **borderline-nitpick**. Here's my adversarial evaluation:

****What genuinely holds (confirmed empirically and by code reading):****

- `import backend.pipeline.segmenters` does pull `torch` and `cv2` into `sys.modules` (empirically True), via the sibling `yolo\_hybrid.py:25` top-level import of `person\_detector`, which does top-level `import torch` / `import cv2` (person_detector.py:26-27).
- `torchvision` and `supervision` (the actual heavy model weights + GPU work) correctly stay lazy ? confirmed NOT in sys.modules.
- The new docstrings at `fusion\_hybrid.py:13` ("never imports torch / touches the GPU") and `:45` ("keeps torch/GPU off the default path") make an unqualified "torch off the default path" claim that is inaccurate at process scope ? torch is already imported before the fusion cue is ever consulted.
- The lazy import at `fusion\_hybrid.py:92` is indeed redundant for the stated purpose (deferring torch): `person\_detector` is already in `sys.modules` after registry import (confirmed True).

****Mitigating factors that pull severity down:****

- The docstring at `fusion\_hybrid.py:91` is precisely scoped ? "never pulls torch/cv2 **through this module**" ? and that qualified claim IS accurate (the fusion_hybrid module itself has no top-level torch/cv2). Only lines 13 and 45 drop the qualifier.
- The **"**touches the GPU**"** half of the claim is fully accurate ? no GPU work or model-weight load happens on the default path (torchvision/supervision stay lazy). So the load-bearing operational guarantee (no GPU, no heavy weights on the default path) is true; only the narrower "imports torch" phrasing is loose.
- It's a docstring-accuracy issue with zero runtime/behavioral impact. The lazy import, while redundant, is harmless and arguably good defensive style.
- The underlying behavior is pre-existing (all three trajectory-hybrid siblings already transitively import torch at registry load), so this is not a regression.

The finding self-assesses as "low" and explicitly disclaims it's pre-existing ? it's honest and accurate, not overstated. It is a legitimate, verifiable doc-accuracy imprecision, just a very minor one.

22. user (2026-06-13T19:16:05.922Z)

Structured output provided successfully

23. assistant (2026-06-13T19:16:13.806Z)

I've completed the adversarial verification.

****Verdict: real=true, severity=low****

The finding holds up against the code. I empirically confirmed (this dev env actually has torch/cv2 installed) that ``import backend.pipeline.segmenters`` pulls ``torch`` and ``cv2`` into ``sys.modules``, while ``torchvision`` and ``supervision`` correctly stay lazy ? exactly as the finding claims. The torch import enters via the sibling ``yolo_hybrid.py:25`` ? ``person_detector.py:26-27`` chain at registry-load time, before the fusion cue is ever consulted. The new docstrings at ``fusion_hybrid.py:13`` and ``:45`` make an unqualified "torch off the default path" claim that's inaccurate at process scope, and the lazy import at ``:92`` is confirmed redundant (person_detector already in sys.modules).

I tried hard to refute it and found real mitigating nuance ? the ``:91`` docstring is precisely scoped ("through this module") and is accurate, and the "touches the GPU" half of the claim is fully correct since the heavy weights/GPU work genuinely stay lazy. That keeps this firmly at low severity. But it's a legitimate, verifiable doc-accuracy imprecision in new (untracked) PR code, not a false-positive ? and the finding's own self-assessment (low, pre-existing behavior) is honest rather than overstated.